Documento de apoyo (Sesion 7, Unidad 2)

Modelado Analitico con dbt

SQL, ELT, pruebas, documentacion y trazabilidad

Javier Andres Valdez Gonzalez / Auxiliar

Seminario de Sistemas 2 — Seccion P

Universidad de San Carlos de Guatemala

Marzo de 2026

Resumen

El modelado analitico moderno requiere transformar datos crudos en modelos confiables, verificables y reutilizables para inteligencia de negocios. En esta sesion se estudia dbt como herramienta central de transformacion ELT: permite escribir modelos en SQL, organizar un proyecto en capas, declarar fuentes, cargar datos semilla, reutilizar logica mediante macros, ejecutar pruebas de calidad, generar documentacion y mantener linaje entre tablas. El documento presenta los fundamentos conceptuales y practicos necesarios para construir modelos analiticos reproducibles, con enfasis en rigor analitico, responsabilidad tecnica, colaboracion mediante control de versiones y trazabilidad de las transformaciones.

Palabras clave: dbt, ELT, modelado analitico, SQL, staging, intermediate, marts, sources, seeds, macros, pruebas de datos, documentacion, linaje, Git.

Índice

1. Introduccion **3**

2. Marco formativo de la sesion **4**
2.1. Area actitudinal: saber ser 4
2.2. Area de conocimiento y habilidades 4

3. Fundamentos de dbt y ELT **5**
3.1. ETL vs ELT 5
3.2. Rol de dbt en el stack moderno de datos 5
3.3. Modelo como unidad de trabajo 6

4. Estructura de un proyecto dbt **7**
4.1. Archivo dbt_project.yml 7
4.2. Models 7
4.3. Sources 7
4.4. Seeds 8
4.5. Macros 8

5. Modelado analitico con dbt **9**
5.1. Capas staging, intermediate y marts 9
5.2. Staging 9
5.3. Intermediate 9
5.4. Marts 10

6. Calidad y documentacion **11**
6.1. Pruebas de datos 11
6.2. Documentacion automatica 11
6.3. Linaje de datos 12

7. Ejecucion y buenas practicas **13**
7.1. Perfiles y ambientes 13
7.2. Ejecucion local 13
7.3. Control de versiones con Git 14

8. Relacion con la practica de la semana 7 **15**

9. Buenas practicas minimas para trabajar con dbt **16**

10. Conclusiones **17**

1. Introduccion

La inteligencia de negocios no depende solamente de visualizar datos en un tablero. Antes de llegar a un dashboard, los datos deben ser extraidos desde sistemas operacionales, cargados en una plataforma analitica, transformados con reglas claras y publicados en tablas comprensibles para usuarios de negocio. Si esa transformacion no es reproducible, documentada y verificable, las metricas pueden volverse ambiguas y las decisiones quedan expuestas a errores dificiles de rastrear.

dbt, abreviatura de *data build tool*, surge precisamente en esa etapa de transformacion. Su proposito es permitir que analistas e ingenieros de datos construyan modelos analiticos mediante SQL, con practicas heredadas del desarrollo de software: modularidad, control de versiones, revisiones, pruebas automatizadas, documentacion y linaje [1]. En lugar de transformar los datos antes de cargarlos, dbt asume que los datos ya estan disponibles en una bodega, lago o plataforma analitica, y ejecuta las transformaciones dentro de esa misma plataforma.

El objetivo de este documento es explicar los temas de la semana 7 desde una perspectiva academica y practica: fundamentos de dbt y ELT, estructura de un proyecto dbt, capas de modelado, pruebas, documentacion, linaje, perfiles, ambientes, ejecucion local y control de versiones con Git. El proposito no es memorizar comandos aislados, sino comprender como dbt ayuda a construir datos confiables para BI.

2. Marco formativo de la sesion

2.1. Area actitudinal: saber ser

El trabajo con dbt no debe entenderse solamente como una actividad tecnica. Al construir modelos analiticos, el estudiante desarrolla habitos profesionales relacionados con calidad, evidencia, colaboracion y trazabilidad. Estos valores son necesarios porque los modelos de datos suelen alimentar reportes, indicadores y decisiones institucionales.

Valor	**Aplicacion en la sesion**
Rigor analitico	Se refuerza al construir modelos de datos consistentes, verificables y reutilizables mediante SQL y dbt.
Responsabilidad tecnica	Se promueve al documentar fuentes, transformaciones y supuestos para evitar decisiones basadas en datos ambiguos.
Colaboracion	Se desarrolla al trabajar con control de versiones, revision de cambios y estandares compartidos de modelado.
Trazabilidad	Se aplica al mantener linaje entre fuentes, modelos intermedios y tablas analiticas finales.

2.2. Area de conocimiento y habilidades

La competencia central de la sesion es construir modelos analiticos reproducibles mediante dbt, SQL y control de versiones, aplicando pruebas, documentacion y buenas practicas de transformacion ELT para preparar datos confiables orientados a BI.

Esta competencia integra el *saber* y el *saber hacer*. En el area de conocimiento, el estudiante comprende los fundamentos de dbt, ELT, estructura de proyecto, capas de modelado, calidad, documentacion y ejecucion. En el area de habilidades, aplica esos conceptos al desarrollar modelos, declarar fuentes, ejecutar pruebas y versionar cambios.

3. Fundamentos de dbt y ELT

3.1. ETL vs ELT

En un proceso ETL, por *Extract, Transform, Load*, los datos se extraen desde las fuentes, se transforman fuera del repositorio analitico y luego se cargan a una bodega o data mart. Este patron fue natural cuando el almacenamiento analitico era costoso y las plataformas tenian menor capacidad para procesar grandes volumenes directamente.

En un proceso ELT, por *Extract, Load, Transform*, los datos se extraen y se cargan primero a una plataforma analitica. La transformacion ocurre despues, usando la capacidad de computo de la bodega o motor de datos. dbt Labs describe ELT como un flujo donde los datos se cargan en una bodega central y luego se transforman dentro de ella para generar modelos utiles para analisis [3].

La diferencia practica puede resumirse asi:

E1. ETL: transforma antes de cargar; suele depender de herramientas externas de integracion.

E2. ELT: carga primero y transforma despues; aprovecha motores analiticos modernos.

E3. dbt: se ubica principalmente en la T de ELT, es decir, en la transformacion gobernada de datos ya cargados.

3.2. Rol de dbt en el stack moderno de datos

dbt no reemplaza todas las piezas de una arquitectura de datos. No es, por si mismo, una herramienta de extraccion desde aplicaciones transaccionales, ni una plataforma de visualizacion. Su rol principal es convertir datos disponibles en la plataforma analitica en modelos consistentes para consumo posterior.

Un stack moderno de datos puede representarse con las siguientes capas:

- **Fuentes operacionales:** bases transaccionales, APIs, archivos, ERPs, CRMs o sistemas de aplicacion.
- **Ingesta y carga:** herramientas que copian o sincronizan datos hacia la plataforma analitica.
- **Transformacion con dbt:** modelos SQL, pruebas, documentacion y linaje.
- **Consumo analitico:** dashboards, reportes, notebooks, modelos predictivos o capa semantica.

Desde esta perspectiva, dbt aporta rigor analitico porque obliga a escribir transformaciones explicitas, revisables y ejecutables. La logica de negocio deja de vivir solamente en consultas sueltas o archivos personales y pasa a formar parte de un proyecto controlado.

3.3. Modelo como unidad de trabajo

En dbt, un modelo es normalmente un archivo SQL que contiene una consulta `select`. dbt compila esa consulta y la materializa en la plataforma de datos como una vista, tabla, tabla incremental u otro tipo de objeto soportado por el adaptador utilizado [13]. Esta idea es central: el modelo no es solamente una consulta temporal, sino un activo de datos que puede versionarse, probarse, documentarse y relacionarse con otros modelos.

Listing 1: Ejemplo simple de modelo dbt en SQL

```
select
    customer_id,
    lower(email) as email,
    created_at,
    current_timestamp as loaded_at
from {{ source('app', 'customers') }}
where customer_id is not null
```

El uso de `source()` indica que la consulta depende de una fuente declarada. Esa dependencia queda registrada en el grafo del proyecto, lo cual permite construir trazabilidad entre datos crudos y modelos finales.

4. Estructura de un proyecto dbt

4.1. Archivo dbt_project.yml

Todo proyecto dbt necesita un archivo `dbt_project.yml`. Este archivo define el nombre del proyecto, la version, el perfil de conexion y configuraciones generales de recursos. La documentacion oficial indica que dbt usa este archivo para entender el contexto del proyecto y como transformar los datos [4].

Una estructura minima puede verse asi:

Listing 2: Estructura conceptual de un proyecto dbt

```
dbt_project.yml
models/
  staging/
  intermediate/
  marts/
seeds/
macros/
tests/
```

Aunque el ejemplo se presenta en un bloque monoespaciado, no representa SQL ejecutable. Su funcion es mostrar la organizacion logica de carpetas que se utilizara durante el modelado.

4.2. Models

La carpeta `models` contiene la logica principal de transformacion. Cada archivo SQL representa una transformacion que dbt puede ejecutar. En proyectos analiticos, los modelos suelen organizarse en capas para separar responsabilidades y facilitar mantenimiento.

La modularidad es importante porque evita consultas gigantes, dificiles de leer y de probar. Un modelo debe tener un objetivo claro: limpiar una fuente, integrar entidades, calcular una metrica o preparar una tabla final para BI. Cuando un modelo intenta resolver demasiadas responsabilidades, aumenta el riesgo de duplicar logica o introducir errores silenciosos.

4.3. Sources

Las fuentes, o `sources`, permiten declarar tablas que ya existen en la plataforma analitica y que son administradas fuera de dbt. Al declararlas, dbt puede documentarlas, probarlas y usarlas como nodos iniciales del linaje [12]. Esto es relevante porque el modelo analitico no debe ocultar de donde provienen los datos.

Una fuente puede incluir descripciones, nombres de tablas, columnas y pruebas basicas. Por ejemplo, se puede validar que una llave primaria no sea nula o que una columna de estado solo contenga valores esperados.

4.4. Seeds

Los `seeds` son archivos CSV que dbt puede cargar a la plataforma de datos como tablas. La documentacion oficial los presenta como datos estaticos que forman parte del proyecto [11]. Son utiles para catalogos pequenos, parametros de referencia, mapeos manuales o tablas auxiliares que no provienen de un sistema operacional.

Un ejemplo universitario podria ser una tabla semilla con codigos de sede, nombres normalizados de categorias o equivalencias entre estados. No conviene usar seeds para grandes volumenes transaccionales; para eso debe existir un mecanismo formal de ingesta.

4.5. Macros

Las macros permiten reutilizar logica mediante Jinja. dbt las usa para evitar repeticion en transformaciones, generar SQL dinamico y encapsular patrones comunes [10]. Una macro debe usarse con criterio: si oculta demasiada logica, el proyecto se vuelve dificil de auditar; si se usa para una regla repetida y bien definida, mejora consistencia.

Por ejemplo, una macro puede estandarizar una expresion de limpieza de texto, calcular rangos de fechas o generar condiciones repetidas para modelos incrementales.

5. Modelado analitico con dbt

5.1. Capas staging, intermediate y marts

Una practica comun en proyectos dbt es dividir los modelos en capas. dbt Labs recomienda pensar el modelado como un conjunto de capas que construyen progresivamente desde fuentes crudas hasta modelos de negocio [2]. En una implementacion academica, esta division ayuda a explicar el proposito de cada transformacion.

C1. Staging: normaliza datos fuente con cambios minimos.

C2. Intermediate: combina, enriquece o prepara calculos que no deben exponerse directamente al negocio.

C3. Marts: publica tablas finales orientadas a procesos de negocio, indicadores y consumo BI.

5.2. Staging

La capa `staging` debe estar cerca de las fuentes. Su objetivo es limpiar nombres de columnas, convertir tipos de datos, aplicar reglas simples de estandarizacion y eliminar detalles tecnicos innecesarios. Normalmente, un modelo staging mantiene la granularidad de la fuente original.

Buenas practicas para esta capa:

- usar nombres claros, por ejemplo `stg_ventas__ordenes`;
- no calcular metricas complejas de negocio;
- conservar llaves necesarias para trazabilidad;
- documentar columnas criticas y supuestos de limpieza;
- aplicar pruebas basicas sobre identificadores, fechas y estados.

5.3. Intermediate

La capa `intermediate` contiene transformaciones que combinan o preparan informacion para los marts. Aqui pueden existir joins entre entidades, deduplicacion, filtros de negocio, calculos auxiliares y estructuras temporales necesarias para simplificar modelos finales.

No todos los proyectos necesitan una capa intermedia extensa. Sin embargo, cuando la logica de negocio crece, esta capa evita que los modelos finales se conviertan en consultas demasiado largas. Tambien permite probar pasos parciales y explicar mejor la transformacion.

5.4. Marts

La capa `marts` representa el resultado analitico final. Sus modelos deben estar orientados a usuarios de negocio: ventas por cliente, inscripciones por periodo, rendimiento academico, inventario por sede, indicadores financieros o cualquier entidad relevante para BI.

En esta capa se recomienda pensar en modelos dimensionales. Kimball y Ross destacan la importancia de hechos, dimensiones, granularidad y conformidad de dimensiones para construir soluciones analiticas comprensibles y reutilizables [14]. dbt no elimina esos fundamentos; los implementa con SQL versionado y automatizable.

Listing 3: Ejemplo conceptual de mart analitico

```
select
    d.fecha,
    c.segmento_cliente,
    p.categoria_producto,
    sum(f.monto_total) as ventas_totales,
    count(distinct f.orden_id) as ordenes
from {{ ref('int_ventas_enriquecidas') }} f
join {{ ref('dim_fecha') }} d on f.fecha_key = d.fecha_key
join {{ ref('dim_cliente') }} c on f.cliente_key = c.cliente_key
join {{ ref('dim_producto') }} p on f.producto_key = p.producto_key
group by
    d.fecha,
    c.segmento_cliente,
    p.categoria_producto
```

El uso de `ref()` declara dependencias entre modelos dbt. Gracias a esas referencias, dbt puede construir el grafo de ejecucion, determinar orden correcto de construccion y mostrar linaje entre capas.

6. Calidad y documentacion

6.1. Pruebas de datos

Las pruebas en dbt son consultas que validan condiciones esperadas sobre modelos, fuentes, semillas u otros recursos. dbt incluye pruebas genericas comunes, como `not_null`, `unique`, `relationships` y `accepted_values` [6]. Tambien permite pruebas singulares escritas como SQL personalizado.

Las pruebas no garantizan que todo analisis sea correcto, pero reducen el riesgo de errores basicos y hacen explicitos los supuestos del modelo. Por ejemplo, si una dimension de clientes debe tener una llave unica, esa regla debe estar codificada como prueba y ejecutarse de forma recurrente.

Listing 4: Ejemplo conceptual de pruebas declaradas en YAML

```
models:
  - name: dim_cliente
    columns:
      - name: cliente_id
        tests:
          - not_null
          - unique
      - name: segmento_cliente
        tests:
          - accepted_values:
              values: ['minorista', 'mayorista', 'corporativo']
```

Aunque el bloque anterior usa estilo monoespaciado, representa configuracion YAML. Se mantiene dentro del mismo formato de codigo para conservar consistencia visual del documento.

6.2. Documentacion automatica

dbt permite documentar modelos, columnas, fuentes y otros recursos dentro de archivos YAML y Markdown. Luego puede generar un sitio de documentacion con descripciones y metadatos del proyecto [8]. Esta documentacion es valiosa porque conecta conocimiento tecnico con comprension de negocio.

Una documentacion adecuada debe responder preguntas como:

- que representa el modelo;

- cual es su granularidad;
- de que fuentes depende;
- que columnas son llaves o metricas;
- que supuestos de transformacion aplica;
- que usuario o proceso consume el resultado.

Documentar no es un tramite posterior. En un proyecto dbt maduro, la documentacion forma parte del desarrollo del modelo y debe revisarse junto con el codigo.

6.3. Linaje de datos

El linaje muestra como un dato se mueve desde una fuente hasta modelos intermedios y tablas finales. dbt construye un grafo de dependencias a partir de funciones como `source()` y `ref()`. Este grafo permite entender que modelos dependen de otros, que tablas se ven afectadas por un cambio y como se relaciona una metrica final con sus fuentes originales [9].

La trazabilidad es especialmente importante en ambientes academicos y empresariales porque permite justificar resultados. Si un indicador cambia, el equipo puede investigar si el cambio proviene de una fuente, una transformacion, una regla de negocio o un error de calidad.

7. Ejecucion y buenas practicas

7.1. Perfiles y ambientes

dbt separa la logica del proyecto de la configuracion de conexion. El archivo `profiles.yml` contiene parametros como motor, host, base de datos, esquema, usuario, credenciales y ambiente de ejecucion [5]. Esta separacion ayuda a ejecutar el mismo proyecto contra ambientes distintos sin cambiar la logica SQL.

En un curso universitario, al menos deben distinguirse:

- **Desarrollo:** ambiente personal para construir y probar modelos.
- **Pruebas:** ambiente controlado para validar cambios antes de publicarlos.
- **Produccion:** ambiente estable consumido por reportes y usuarios finales.

La responsabilidad tecnica implica no mezclar datos experimentales con modelos que alimentan decisiones reales. Tambien implica no guardar credenciales sensibles dentro del repositorio.

7.2. Ejecucion local

Un flujo local basico con dbt suele incluir validacion de conexion, ejecucion de modelos, pruebas y generacion de documentacion. Los comandos exactos pueden variar segun adaptador y plataforma, pero la secuencia conceptual es estable:

Listing 5: Flujo local basico de trabajo con dbt

```
dbt debug
dbt seed
dbt run
dbt test
dbt docs generate
dbt docs serve
```

`dbt debug` ayuda a validar configuracion; `dbt seed` carga datos semilla; `dbt run` construye modelos; `dbt test` ejecuta pruebas; `dbt docs generate` produce artefactos de documentacion; y `dbt docs serve` permite visualizarla localmente [7].

7.3. Control de versiones con Git

dbt se beneficia del control de versiones porque los modelos son archivos de texto: SQL, YAML y Markdown. Esto permite trabajar con ramas, revisar cambios mediante *pull requests*, comparar versiones y recuperar el historial de decisiones tecnicas. dbt Labs resalta que la versionacion, las pruebas y la documentacion son parte de la disciplina de ingenieria aplicada a datos [1].

Buenas practicas minimas:

- crear una rama por cambio funcional;
- revisar SQL, YAML, pruebas y documentacion en conjunto;
- exigir que `dbt run` y `dbt test` pasen antes de integrar cambios;
- escribir mensajes de commit que expliquen el impacto analitico;
- evitar subir credenciales, archivos locales de perfil o datos sensibles.

8. Relacion con la practica de la semana 7

La practica de esta semana debe orientar al estudiante a construir un proyecto dbt pequeno pero completo. El objetivo no es solamente ejecutar comandos, sino observar como se construye una cadena reproducible desde fuentes hasta modelos finales.

Una practica recomendable puede seguir esta secuencia:

P1. declarar una fuente con datos cargados previamente;

P2. crear modelos `staging` para limpiar nombres, tipos y valores;

P3. construir un modelo `intermediate` que integre entidades;

P4. publicar uno o mas modelos `marts` orientados a BI;

P5. agregar pruebas de unicidad, nulidad, relaciones y valores aceptados;

P6. documentar modelos y columnas principales;

P7. generar documentacion y revisar el linaje;

P8. versionar el cambio con Git.

Este recorrido refuerza las areas actitudinales de la sesion. El rigor analitico aparece al validar modelos; la responsabilidad tecnica se evidencia al documentar fuentes y supuestos; la colaboracion se desarrolla mediante Git y revision de cambios; y la trazabilidad se materializa en el linaje entre fuentes, modelos intermedios y tablas finales.

9. Buenas practicas minimas para trabajar con dbt

Tema	Practica recomendada
Rigor analitico	Definir granularidad, llaves y metricas antes de publicar un mart para usuarios de negocio.
Sources	Declarar fuentes en YAML, documentarlas y aplicar pruebas basicas sobre columnas criticas.
Staging	Mantener transformaciones simples, cercanas a la fuente y con nombres consistentes.
Intermediate	Usar esta capa para joins, deduplicacion y preparacion de logica que no debe exponerse directamente.
Marts	Disenar tablas finales orientadas a procesos de negocio, indicadores y consumo BI.
Pruebas	Automatizar validaciones de nulidad, unicidad, integridad referencial y valores permitidos.
Documentacion	Documentar significado, granularidad, supuestos y dependencias de modelos y columnas.
Git	Revisar cambios mediante ramas y commits claros; ejecutar pruebas antes de integrar al proyecto principal.

10. Conclusiones

dbt permite aplicar disciplina de ingenieria al modelado analitico. Su valor no esta solamente en ejecutar SQL, sino en convertir transformaciones dispersas en un proyecto reproducible, documentado, probado y versionado. Esta forma de trabajo mejora la confianza en los datos y reduce la ambiguedad de las metricas usadas en inteligencia de negocios.

La separacion entre `staging`, `intermediate` y `marts` ayuda a organizar responsabilidades. Las fuentes y semillas permiten controlar el punto de partida; los modelos expresan transformaciones; las pruebas verifican supuestos; la documentacion comunica significado; y el linaje conserva trazabilidad. En conjunto, estos elementos permiten construir datos confiables para BI y preparan al estudiante para colaborar en proyectos modernos de analitica de datos.

Referencias

[1] dbt Labs. «dbt explained,» visitado 30 de ago. de 2026. dirección: https://www.getdbt.com/blog/dbt-explained

[2] dbt Labs. «Staging models best practices and limiting view runs,» visitado 30 de ago. de 2026. dirección: https://www.getdbt.com/blog/staging-models-best-practices-and-limiting-view-runs

[3] dbt Labs. «Getting started with an ELT pipeline,» visitado 30 de ago. de 2026. dirección: https://www.getdbt.com/blog/elt-pipeline

[4] dbt Labs. «About dbt projects,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/build/projects

[5] dbt Labs. «Connection profiles,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/core/connect-data-platform/profiles.yml

[6] dbt Labs. «Data tests,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/build/data-tests

[7] dbt Labs. «dbt commands,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/reference/dbt-commands

[8] dbt Labs. «dbt Docs,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/build/documentation

[9] dbt Labs. «Getting started with data lineage,» visitado 30 de ago. de 2026. dirección: https://www.getdbt.com/blog/getting-started-with-data-lineage

[10] dbt Labs. «Jinja and macros,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/build/jinja-macros

[11] dbt Labs. «Seeds,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/build/seeds

[12] dbt Labs. «Sources,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/build/sources

[13] dbt Labs. «SQL models,» visitado 30 de ago. de 2026. dirección: https://docs.getdbt.com/docs/build/sql-models

[14] R. Kimball y M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3.ª ed. Wiley, 2013.